

UNIVERSITAT DE BARCELONA

FUNDAMENTAL PRINCIPLES OF DATA SCIENCE MASTER'S
THESIS

**Logic Tensor Networks for
Knowledge-Infused Learning:
Foundations, Experiments, and
Applications**

Author:

Quim GUIOT CUSIDÓ

Supervisor:

Dr. Oriol PUJOL VILA

*A thesis submitted in partial fulfillment of the requirements
for the degree of MSc in Fundamental Principles of Data Science
in the*

Facultat de Matemàtiques i Informàtica

June 27, 2026

UNIVERSITAT DE BARCELONA

Abstract

Facultat de Matemàtiques i Informàtica

MSc

Logic Tensor Networks for Knowledge-Infused Learning: Foundations, Experiments, and Applications

by Quim GUIOT CUSIDÓ

This thesis investigates Logic Tensor Networks (LTN) as a neurosymbolic framework for knowledge infused learning, focusing on their theoretical underpinnings, practical behavior, and applicability to real world tasks. We first present a self contained exposition of Real Logic, detailing how symbols are grounded into tensors, how fuzzy logical operators are instantiated in a differentiable setting, and how learning is framed as maximizing the satisfiability of a Real Logic theory. Building on this foundation, we design a controlled 1D regression problem where a neural model predicts a person's weight from their height under different logical configurations. Through three experiments (baseline regression, a favorable monotonicity axiom, and a deliberately misleading monotonicity axiom) we study how prior knowledge encoded as axioms affects in distribution performance, out of distribution generalization, and the truth degree of queries. The results show that well aligned axioms can improve both regression accuracy and robustness, while incorrect axioms systematically degrade performance, highlighting LTN as a sensitive interface between data and prior knowledge. Finally, we explore a real life recommender system case study, where LTN are used to mitigate data sparsity by injecting genre based user preferences into a matrix factorization pipeline. Altogether, this work illustrates both the promise and the risks of knowledge infused learning with LTN.

The following link will take the lector to the GitHub Repository containing the performed and developed experiments discussed on chapter 4: **My GitHub Repository** . URL to copy paste on your browser: <https://github.com/srguiot14/Logic-Tensor-Networks-for-Knowledge-Infused-Learning-Foundations-Experiments-and-Applications>

Acknowledgements

Throughout the development of this thesis, we used AI on the following tasks.

First of all, I want to show my gratitude to my supervisor and advisor Dr. Oriol Pujol i Vila, who introduced me to the LTN world and has guided me through the development of this thesis, from the choice of the topic to the last corrections, for the last 8 months.

Secondly, I want to thank my mother and father Teresa Antònia and Miquel, as well as my brother Miquel, for always giving me their help and support. They made me the person I am and allowed me to be in the position where I am, able to achieve this goal. Therefore, any achievement of myself also belongs to them.

Statement on the use of AI

Throughout the development of this thesis, we used AI on the following tasks.

First of all, AI has been used to code all notebooks of the different experiments performed. Coding tasks in which it has been used compress how to structure the code, how syntax is written, performing debugging tasks... However, deciding the experiments, measures, axioms, logical operators configuration... has all been my decision as author.

Secondly, AI has been used on the process of writing down this report. The first aspect is regarding $\text{\LaTeX}$ syntax and debugging errors. The second one is getting assisted on tasks like summarizing the content of the report in order to write sections like the introduction, conclusions and the abstract. Additionally, since the first version of this thesis exceeded a little bit the extension limits required, I asked AI also to help me compressing some particular sections through "optimizing" the writing.

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
2 Background/State of the Art	3
3 Logic Tensor Networks	5
4 Experimental setting	9
5 Real Life Application: Recommender System	15
6 Conclusions	19
Bibliography	21

Chapter 1

Introduction

Motivation

Machine learning systems excel at extracting patterns from large datasets and perform quick decision-making, but often struggle to generalize outside training distribution, are data inefficient and don't build a comprehensible world model.

Symbolic AI, which is based on rich and high-level representations of the world (i.e. logical representations), has achieved success at tasks like logical inference or theorem proving. However, they struggle when dealing with noisy, inaccurate data.

Neurosymbolic learning arises as an attempt to combine these paradigms, aiming to achieve models both capable of learning from complex data itself and reason with complex representations of knowledge. Moreover, data itself only allows models to obtain associations and correlations, not having access to some model of the world beyond this data. Neurosymbolic AI also provides a solution to this issue, providing through rich knowledge a model of the world that complements data. LTN have been recently introduced as a new neurosymbolic learning paradigm able to provide a practical instantiation of this idea. As can be gathered from this approach, how symbols (constants, functional symbols...) are interpreted as real data is a major challenge, which will be discussed in the following sections.

Goals and contributions

This thesis aims to introduce LTN, explain their theoretical foundations, illustrate them through a simple example of a 1D linear regressor, test their usefulness and risks, and make a demonstration of their different applications.

The three main contributions of the thesis are the following: (i) it provides a clear, self-contained exposition of LTN's Real Logic, grounding mechanisms and the loss based on satisfaction used to train parametric groundings; (ii) through a toy regression study, it shows how different axioms (regression, local monotonicity, and misleading constraints) affect learning, generalization and out-of-distribution behavior; and (iii) this thesis also discusses the potential of LTN applied at knowledge transfer tasks through presenting a result achieved on a recently published paper.

Layout

Chapter 2 reviews background on fuzzy semantics, differentiable logic and related neurosymbolic work. Chapter 3 formalizes LTN and the Real Logic operators used in this thesis. Chapter 4 describes the experimental framework and the toy regression experiments that illustrate LTN mechanics and the obtained results. Chapter 5 presents the recommender case study, experimental setup, and comparative results against a baseline Matrix Factorization. The thesis closes with conclusions, limitations and directions for future work.

Chapter 2

Background/State of the Art

Knowledge Infusion was first introduced in Valiant, 2006, when Leslie Valiant mentioned that the biggest step we could take in order to make computers more useful for humans would be giving them the ability to acquire and manipulate what we commonly denote as common sense. This proposal is what is called knowledge Infusion.

This first approach proposed a theoretical basis for knowledge infusion in terms of robust logic, which distinguished knowledge infusion from other concurrent approaches to reconciling learning and reasoning in three main aspects. Firstly, it provided a common semantics for both the learning and reasoning. Secondly, it gave priority to the existence of polynomial time and data efficient algorithms for all aspects of the process. And finally, it had the goal of extracting from data knowledge that would be useful in scenarios not foreseen at the time of the extraction.

The first steps towards achieving this goal was defining an architecture for systems to achieve it on Valiant, 2000b. Afterwards, robust logic was introduced in Valiant, 2000a as a more foundational work required to define properly a semantics. Finally, Valiant, 2006 formalized the concept of knowledge infusion and started pointing at the theoretical issues and challenges that must be dealt with; beginning this way the knowledge infusion era on machine learning.

Logic Tensor Networks arose for the first time on Serafini, Donadello, and Garcez, 2017, when they were first introduced together with Real logic, and, since then, have been of the most prominent paradigms in this area. They have been applied so far on AI tasks involving perception, learning and reasoning about relational knowledge, like semantic image interpretation, learning an embedding of concepts into a latent real space or performing reasoning about ontological knowledge.

Finally, Badreddine et al., 2020 provided a thorough description of the full formalism and several extensions of LTN, as well as showing how they can be applied to solving many ML tasks with the help of logical knowledge.

Chapter 3

Logic Tensor Networks

In this section, we aim to define the basic theoretical concepts required to understand how LTN work, which are the elements that compose them and what is the general architecture of any LTN. We define LTN as follows:

Definition 1. *Logic Tensor Networks (LTN) are a neurosymbolic framework and computational model that supports learning and reasoning about data using a knowledge base composed of a set of first order logic (FOL) axioms as the objective of a neural network model.*

The basis of LTN is this first order logical language in which all their elements are defined. Precisely, the logical language that they implement is Real Logic and formalizes in the following definition:

Definition 2. *Real Logic is a fully differentiable logical language defined in a first-order language $\mathcal{L}$ with a signature that contains a set $\mathcal{C}$ of constant symbols (objects), a set $\mathcal{F}$ of functional symbols, a set $\mathcal{P}$ of relational symbols (also called predicates) and a set $\mathcal{X}$ of variable symbols.*

From the previous definition, we can see that the objects (variables and constants) of $\mathcal{L}$ can be of different types, while functions and predicates are typed. Therefore, there is a need to assign some type to the elements of $\mathcal{L}$ that leads to the following definition.

Definition 3. *The set $\mathcal{D}$ is defined as the set of symbols containing the types of the elements of $\mathcal{L}$, and is called domain symbols. These types are assigned through the functions $\mathbf{D}$: $\mathcal{X} \cup \mathcal{C} \rightarrow \mathcal{D}$, $\mathbf{D}_{in} : \mathcal{F} \cup \mathcal{P} \rightarrow \mathcal{D}^*$ (where $\mathcal{D}^*$ denotes the set of all finite sequences of symbols in $\mathcal{D}$) and $\mathbf{D}_{out} : \mathcal{F} \rightarrow \mathcal{D}$. Intuitively, $\mathbf{D}()$ returns the domain of a variable x or a constant c , $\mathbf{D}_{in}()$ returns the domains of the arguments of a function f or a predicate p , and $\mathbf{D}_{out}()$ returns the range of a function symbol (the domain of its output).*

Combining variables, constants, and functions of $\mathcal{L}$ we are able to construct terms (t); and applying a predicate symbol to an appropriate number of terms we build atomic formulas that evaluate to a number in $[0, 1]$, representing the degree of truth of such statement.

This evaluation of the $[0, 1]$ value instead of the true or false classical logic is crucial in LTN and is caused by the type of logic implemented. Since LTN are intended to be applied in real world scenarios, classical logic approaches in which the logical statements are absolutely false (value 0) or true (value 1) are not feasible. In real life, exceptions may appear, and commonly known "general rules" or any kind of formula can be partially true; therefore, some measure of fuzzy truth degree is required. This is the reason why LTN adopts fuzzy logic semantics.

Groundings

As has been mentioned in the introduction, the interpretation of symbols in real data is a major challenge in neurosymbolic learning; in this thesis, we will refer to this interpretation as grounding (denoted by $\mathcal{G}$), to emphasize that Real Logic symbols are grounded in real valued features. A grounding for a logical language $\mathcal{L}$ on the set of domains $\mathcal{D}$ provides the interpretation of both the domain symbols in $\mathcal{D}$ and the non-logical symbols in $\mathcal{L}$. Firstly, the grounding of a domain is defined as:

Definition 4. A grounding $\mathcal{G}$ associates to each domain $D \in \mathcal{D}$ a set $\mathcal{G}(D) \subseteq \bigcup_{n_1, \dots, n_d \in \mathbb{N}^*} \mathbb{R}^{n_1 \times \dots \times n_d}$. $\forall D_1, \dots, D_n \in \mathcal{D}^*, \mathcal{G}(D_1 \dots D_n) = \times_{i=1}^n \mathcal{G}(D_i)$.

Whereas the grounding of $\mathcal{L}$ is defined as:

Definition 5. A grounding $\mathcal{G}$ of $\mathcal{L}$ is a function defined on the signature of $\mathcal{L}$ that satisfies the following conditions

1. $\mathcal{G}(c) = d \in \mathcal{G}(\mathbf{D}(x))$ for every constant symbol $c \in \mathcal{C}$, where d is a tensor of real numbers.
2. $\mathcal{G}(x) = \langle d_1 \dots d_k \rangle \in \times_{i=1}^k \mathcal{G}(\mathbf{D}(x))$ for every variable symbol $x \in \mathcal{X}$, with $k \in \mathbb{N}_0^+$; where d_i is a tensor of real numbers $\forall i$.
3. $\mathcal{G}(f) \in \mathcal{G}(\mathbf{D}_{in}(f)) \rightarrow \mathcal{G}(\mathbf{D}_{out}(f))$ for every function symbol $f \in \mathcal{F}$.
4. $\mathcal{G}(p) \in \mathcal{G}(\mathbf{D}_{in}(p)) \rightarrow [0, 1]$ for every predicate symbol $p \in \mathcal{P}$.

If a grounding depends on a set of parameters θ , we denote it as $\mathcal{G}_\theta(\cdot)$ or $\mathcal{G}(\cdot|\theta)$ interchangeably.

Intuitively, $\mathcal{G}$ associates to any term t of $\mathcal{L}$ a tensor of real numbers which represent the numeric features of the objects denoted by t ; whereas, it associates to any formula ϕ of $\mathcal{L}$ a real number in the interval $[0, 1]$ which represents the system's degree of confidence in the truth of ϕ .

Example 1: Let $\mathcal{L}$ be a logical language containing the set of domains $\mathcal{D} = \{A, B\}$; the variables x, y ; the function f ; and the predicate p ; where $\mathbf{D}(x) = A$, $\mathbf{D}(y) = B$, $\mathbf{D}_{in}(f) = AB$, $\mathbf{D}_{out}(f) = B$, and $\mathbf{D}(p) = AB$. Then: $\mathcal{G}(A) = \mathbb{R}^-$, $\mathcal{G}(B) = \mathbb{R}$, $\mathcal{G}(x) = \langle a_1, a_2 \rangle$, $\mathcal{G}(y) = \langle b_1, b_2 \rangle$, $\mathcal{G}(f) : x, y \rightarrow x \cdot y$, and $\mathcal{G}(p) : x, y \rightarrow \sigma(x + y)$. Where $a_1, a_2 \in \mathbb{R}^-$; $b_1, b_2 \in \mathbb{R}$; and $\mathcal{G}(f)$ and $\mathcal{G}(p)$ are applied element-wise:

$$\mathcal{G}(f(x, y)) = \begin{pmatrix} a_1 \cdot b_1 & a_1 \cdot b_2 \\ a_2 \cdot b_1 & a_2 \cdot b_2 \end{pmatrix} \quad \mathcal{G}(p(x, y)) = \begin{pmatrix} \sigma(a_1, b_1) & \sigma(a_1, b_2) \\ \sigma(a_2, b_1) & \sigma(a_2, b_2) \end{pmatrix}$$

Making a brief forecast of chapter 4, in a weight prediction based on height scenario, variables would be x and y for the samples and the weights respectively, with $\mathbf{D}(x) = \text{samples}$, $\mathbf{D}(y) = \text{weights}$, $\mathcal{G}(\text{samples}) = \mathcal{G}(\text{samples}) = \mathbb{R}$ and $\mathcal{G}(x) = \mathcal{G}(y) = \mathbb{R}^m \times 1$, where m is the number of examples on the sample.

Logical operators

The other element required to construct logical statements are the logical operators, both connectives and quantifiers. The semantics of the connectives is defined according to the semantics of first-order fuzzy logic; precisely, conjunction ($\wedge$), disjunction ($\vee$), implication ($\rightarrow$) and negation ($\neg$) are associated with t-norms T , t-conorm S ,

fuzzy implication I , and fuzzy negation N , respectively¹. On the other hand, the semantics of quantifiers (exists ($\exists$) and for all ($\forall$)) is defined with the use of aggregation, picking a suitable symmetric and continuous operator $\text{Agg} : \bigcup_{n \in \mathbb{N}} [0, 1]^n \rightarrow [0, 1]$.

However, not all first-order fuzzy logic semantics are equally suited for gradient-descent optimization since many can lead to exploding or vanishing gradients. The best performing symmetric configuration is presented in the following definition.

Definition 6. *The following subset of Real Logic is restricted to the product configuration is called Product Real Logic. Given a and b two truth-values in $[0, 1]$:*

$$\neg : N_S(a) = 1 - a \quad (3.1)$$

$$\wedge : T_P(a, b) = ab \quad (3.2)$$

$$\vee : S_P(a, b) = a + b - ab \quad (3.3)$$

$$\rightarrow : I_R(a, b) = 1 - a + ab \quad (3.4)$$

$$\exists : A_{pM}(a_1, \dots, a_n) = \left(\frac{1}{n} \sum_{i=1}^n a_i^p \right)^{\frac{1}{p}}, \quad p \geq 1 \quad (3.5)$$

$$\forall : A_{pME}(a_1, \dots, a_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - a_i)^p \right)^{\frac{1}{p}}, \quad p \geq 1 \quad (3.6)$$

Learning with LTN

As we have seen so far, a Real Logic knowledge-base summarizes in three components; a first one that describes knowledge through the groundings of symbols (domains, functions...), a second one consisting in a set of closed logical formulas describing general knowledge and factual propositions, and a third one encoded in the operators and hyperparameters used to evaluate each formula. This notion formalizes in the following definition:

Definition 7. *A theory of Real Logic is a triple $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}(\cdot|\theta), \Theta \rangle$, where $\mathcal{K}$ is a set of closed first-order logic formulas defined on the set of symbols $S = D \cup X \cup C \cup F \cup P$; $\mathcal{G}(\cdot|\theta)$ is a parametric grounding for all the symbols $s \in S$ and all logical operators; and $\Theta = \{\Theta_s\}_{s \in S}$ is the hypothesis space for each set of parameters θ_s associated with symbol s .*

It follows from this definition that the process of learning in a Real Logic theory is associated with searching and applying the set of values of the parameters θ from Θ that maximize the satisfaction of the formulas in $\mathcal{K}$. A Real Logic theory with a specific set of learned parameter values is called grounded theory and is denoted by $\langle \mathcal{K}, \mathcal{G}_\theta \rangle$. The optimization problem that constitutes the learning of a Real Logic theory is defined by aggregating the truth-values of all the formulas in $\mathcal{K}$ by selecting a formula aggregating operator $\text{SatAgg} : [0, 1]^* \rightarrow [0, 1]$. This notion formalizes in the following two definitions:

Definition 8. *The satisfiability of a theory $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}_\theta \rangle$ with respect to the aggregating operator SatAgg is defined as $\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi)$.*

¹Since it is out of the scope of this thesis, we will not define this concepts. If any lector is curious, can check Appendix B of Badreddine et al., 2020

Definition 9. Given a Real Logic theory $\mathcal{T} = (\mathcal{K}, \mathcal{G}(\cdot|\theta), \Theta)$, learning is the process of searching for the set of parameter values θ^* that maximize the satisfiability of $\mathcal{T}$ w.r.t. a given aggregator:

$$\theta^* = \underset{\theta \in \Theta}{\operatorname{argmax}} \underset{\phi \in \mathcal{K}}{\operatorname{SatAgg}} \mathcal{G}_\theta(\phi) \quad (3.7)$$

Architecture

So far, we have seen the different components of an LTN, but still have not discussed the general architecture that they constitute once all of them are integrated together.

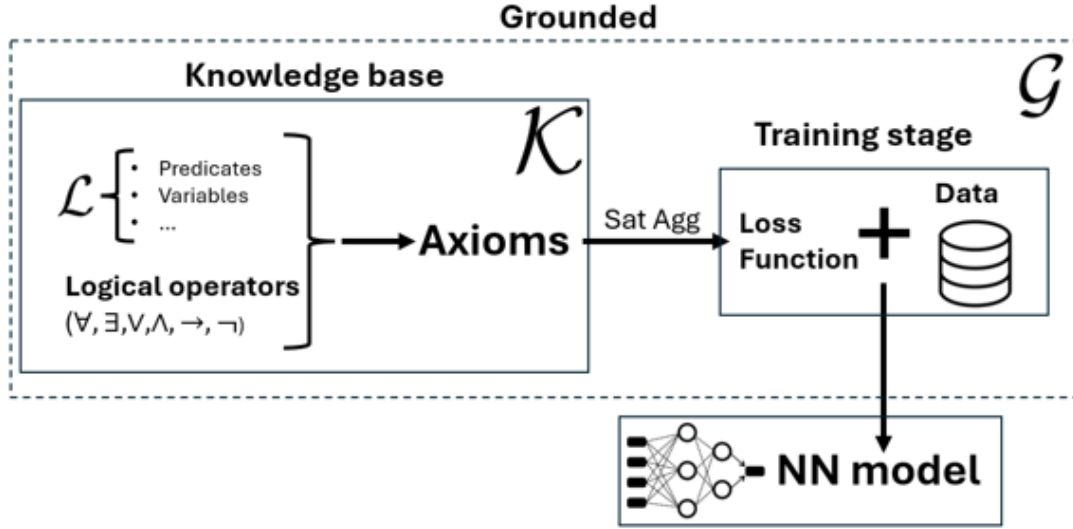


FIGURE 3.1: General architecture schema of a LTN.

The goal of the whole process is ending up constructing a set of logical axioms that represent some knowledge we want to inject to the model, and that we want to be maximally satisfied.

In order to do so, first of all this axioms are constructed combining the logical operators with the predicates, variables and other elements of the logical language $\mathcal{L}$ that we decided. Once all has been designed and formulated theoretically, we are able to combine it with data and "translate to the machine language" by grounding it as we find more convenient. Notice that the choice itself of what operators configuration we implement, what grounding assign to the functions and predicates... also encode knowledge. Once this knowledge base is stated, all the axioms are combined to design the objective function the model is going to optimize (the aggregating operator SatAgg). Finally, a neural network model (i.e. a MLP) is trained on the training data, using the above mentioned objective function as the metric.

Figure 3.1 show a schematic representation of this architecture.

Chapter 4

Experimental setting

In order to illustrate on a practical scenario all the elements of LTN, we performed a toy problem consisting on a 1-D linear regressor. This section aims to develop a controlled environment on which we can show through a very simple problem how LTN work, as well as what shape takes each of its elements. This problem consists on trying to predict the weight of a person using as independent feature its height.

Data

First of all, regarding the data, we used as dataset a 300 examples sample generated on the range $x \in [1.4 \text{ m}, 2.0 \text{ m}]$ according to the formula:

$$y = 0.9 \cdot (100 \cdot x) + 5 \cdot \sin(3 \cdot x), \quad (4.1)$$

We added some random noise to y according to a normal distribution with standard deviation equal to 3 and mean equal to the value of y at each case. Out of the 300 examples, 80% have been used as training whereas the remaining 20% as test.

Moreover, in order to perform an additional measurements, we generate out of distribution data (OOD) using the same equation (4.1). We generated 400 additional examples with height values in the intervals $[1.0, 1.4]$ and $[2.0, 2.3]$. This will be used to measure the accuracy in this extended domain.

Models

In order to predict a persons weight based on its height, we are using a linear regressor as a classifier while implementing LTN. We model it by defining f^* as a learnable function whose parameter values are constrained by data. We picked as f^* a multi-layer perceptron (MLP) deep model. In particular, it is a 3-layer MLP with an input of dimension 1 and 2 hidden layers of 8 neurons with elu activation function and ending in one neuron with a linear output layer without activation function. Additionally, since every regression task requires a notion of equality, we turned constraint $f(x_i) = y_i$ into a smooth optimization problem by modeling the symbol "=" in a smooth way through the predicate eq that we will define on a subsection below.

This approach constitutes the baseline model. As an additional modification, we added a second axiom in which we stimulate the model to have locally monotonously behavior. This will be later modified to show the role of axioms in LTN.

Metrics

Regarding the metrics that we have computed in order to measure the performance of each model, we have measured, for each experiment, the satisfaction degree of each axiom as well as the aggregated loss function.

In experiment 1, we only use the regression axiom. As a consequence, the loss function of this setting is composed only of the degree of satisfaction of this one. In experiment 1 we used operator A_{pME} with $p = 4$ as aggregator for the $\forall$ syntax, whereas for other operators we implemented the Product Real Logic configuration.

Thus, experiment 1 has the following satisfaction degree optimization problem:

$$\text{sat}_{\text{reg}} = 1 - \left(\frac{1}{N} \sum_{i=1}^N (1 - \exp(-0.01|f(x_i) - y_i|))^4 \right)^{1/4} \quad (4.2)$$

And the following loss function:

$$L = 1 - \text{sat}_{\text{reg}} = \left(\frac{1}{N} \sum_{i=1}^N (1 - \exp(-0.01|f(x_i) - y_i|))^4 \right)^{1/4} \quad (4.3)$$

On the other hand, experiments 2 and 3 have both the regression and the monotonicity axiom. Consequently, there are two satisfaction degrees that must be optimized and that make up the loss function. For both of them, we used operator A_{pME} with $p = 4$ as aggregator for the $\forall$ syntax on the regression axiom, operator A_{pM} with $p = 1$ as aggregator for the $\forall$ syntax on the monotonicity axiom and queries 1 and 2, and Product Real Logic configuration for other operations.

Thus, experiments 2 and 3 have the satisfaction degree optimization problems:

$$\text{sat}_{\text{reg}} = 1 - \left(\frac{1}{N} \sum_{i=1}^N (1 - \exp(-0.01|f(x_i) - y_i|))^4 \right)^{1/4} \quad (4.4)$$

$$\text{sat}_{\text{mon}} = \frac{1}{M} \sum_{(x_1, x_2)} (1 - g(x_1, x_2) + g(x_1, x_2)h^*(a, b)) \quad , \quad (4.5)$$

$$\text{where } h^*(a, b) = \begin{cases} h(f(x_1), f(x_2)) = \sigma(-20(f(x_1) - f(x_2))) & \text{on exp. 2,} \\ h(f(x_2), f(x_1)) = \sigma(-20(f(x_2) - f(x_1))) & \text{on exp. 3.} \end{cases}$$

And, combining them both weighted by the parameter λ (which we set to 0.05), it is obtained the aggregated satisfaction function:

$$\text{sat}_{\text{total}} = (1 - \lambda)\text{sat}_{\text{reg}} + \lambda\text{sat}_{\text{mon}} \quad (4.6)$$

And therefore the loss function on these two experiments is the following:

$$L = 1 - \text{sat}_{\text{total}} = (1 - \lambda)(1 - \text{sat}_{\text{reg}}) + \lambda(1 - \text{sat}_{\text{mon}}) \quad (4.7)$$

Additionally, we also computed the classical RMSE to measure the performance of the model on all 3 experiments:

$$\varepsilon_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (f^*(x_i) - y_i)^2} \quad (4.8)$$

Finally, we also measured the degree of truth that some statements about the dataset hold. In order to do so, we deployed the following 2 queries:

$$\forall x, x < 1.75 \Rightarrow f(x) < 70 \quad \equiv \quad \forall x, g(1.75, x) \Rightarrow h(70, f(x)) \quad (4.9)$$

$$\forall x, x > 1.65 \Rightarrow f(x) > 75 \equiv \forall x, g(x, 1.65) \Rightarrow h(f(x), 75) \quad (4.10)$$

Which check the degree of truth of the statements “every subject who measures less than 1.75 meters, weights less than 70 pounds” and “every subject who measures more than 1.65 meters, weights more than 75 pounds” respectively.

Similarly to the monotonicity axiom, this truth evaluation are performed measuring the satisfaction degree of the above expressions. Precisely, the evaluated expressions are the following:

$$\text{sat}_{q1} = \frac{1}{M} \sum_{i=1}^N (1 - g(1.75, x_i) + g(1.75, x_i)h(70, f(x_i))) \quad , \quad (4.11)$$

$$\text{sat}_{q2} = \frac{1}{M} \sum_{i=1}^N (1 - g(x_i, 1.65) + g(x_i, 1.65)h(f(x_i), 75)) \quad , \quad (4.12)$$

for queries 1 (Eq. 4.9) and 2 (Eq. 4.10) respectively.

Parameters

As stated above, on all 3 experiments we used the Product Real Logic configuration, picking $p = 2$ for the $\exists$ operator and $p = 4$ for the $\forall A_{pME}$ operator used on the regression axiom as hyperparameters values. On experiments 2 and 3, we defined an additional $\forall$ just for the monotonicity axiom such that it was a less strict statement than the regression one and for queries 1 and 2. This monotonicity operator $\forall$ is defined as $A_{pM}(a_1, \dots, a_n)$ picking $p = 1$ as the hyperparameter value.

Regarding computational and training configuration, all 3 experiments have been deployed using a batch size of 128, a learning rate of 0.0005, 1500 training epochs and a train/test data splitting of 80-20. Moreover, for experiment 2 and 3 specifically, when computing the locally monotonicity axiom satisfaction value, we took into account 3 previous and 3 posterior points for each point. Additionally, the λ parameter that determines the weight of each component of the loss function (the satisfaction of the regression and monotonicity axiom) is set to 0.05. Meaning that regression axiom has a weight of 0.95 whereas monotonicity axiom has a 0.05 weight.

Language and groundings

Finally, before moving to describing the actual experiments, we expose the language and grounding of the components of the problem that LTN use:

Domains:

samples, denoting the subjects and their heights.
weights, denoting the subjects weights.

Variables:

x for the samples. $\mathbf{D}(x) = \text{samples.}$
 y for the weights. $\mathbf{D}(y) = \text{weights.}$

Functions:

$f^*(x)$, the regression function
to be learned. $\mathbf{D}_{\text{in}}(f^*) = \text{samples.}$
 $\mathbf{D}_{\text{out}}(f^*) = \text{weights.}$

Predicates:

$\text{eq}(y_1, y_2)$, a smooth equality predicate that measures how similar y_1 and y_2 are.

$g(y_1, y_2)$, a smooth greater than or equal predicate that evaluates how strongly the relation $y_1 \geq y_2$ holds.

$h(f(y_1), f(y_2))$, a smooth greater than or equal predicate that evaluates how strongly the model output satisfies $f(y_1) \geq f(y_2)$.

Axioms:

$$\forall_{reg} \text{Diag}(x, y) \text{eq}(f^*(x), y) \quad (4.13)$$

$$\forall_{mon} (x_1, x_2) \in N_3(x) (g(x_1, x_2) \Rightarrow h(f(x_1), f(x_2)))^{**} \quad (4.14)$$

$$\forall_{mon} (x_1, x_2) \in N_3(x) (g(x_1, x_2) \Rightarrow h(f(x_2), f(x_1)))^{**} \quad (4.15)$$

where 4.14 is used on experiment 2 and 4.15 on experiment 3.

**Where $N_3(x)$ denotes the set of pairs of points s.t. x_1 is one of the 3 nearest previous or 3 posterior nearest points to x_2 .

Queries:

$$\forall_{reg} x, (g(1.75, x) \Rightarrow h(70, f(x))) \quad (4.16)$$

$$\forall_{reg} x, (g(x, 1.65) \Rightarrow h(f(x), 75)) \quad (4.17)$$

Groundings:

$$\mathcal{G}(\text{samples}) = \mathbb{R}.$$

$$\mathcal{G}(\text{weights}) = \mathbb{R}.$$

$\mathcal{G}(x) = \mathbb{R}^m \times 1$, $\mathcal{G}(y) = \mathbb{R}^m \times 1$. Where m denotes the number of examples in the sample. It refers to the same number of examples for both x and y due to the one-to-one correspondence stated on the regression axiom 4.13 using Diag.

$\mathcal{G}(\text{eq}(\mathbf{u}, \mathbf{v})) = \exp(-\alpha|u - v|)$. Where $\alpha \in \mathbb{R}$ is an hyperparameter that calibrates the degree of equality strictness. In our experiments, we used $\alpha = 0.01$.

$\mathcal{G}(g(x_1, x_2)) = \sigma(-\beta(x_1 - x_2)) = \frac{1}{1 + \exp(\beta(x_1 - x_2))}$. Where σ is the sigmoid function and $\beta \in \mathbb{R}$ is an hyperparameter that calibrates the degree of abruptness of the transition. In our experiments, we used $\beta = 20$.

$$\mathcal{G}(h(x_1, x_2)) = \sigma(-\beta(f(x_1) - f(x_2))) = \frac{1}{1 + \exp(\beta(f(x_1) - f(x_2)))}.$$

$$\mathcal{G}(f^*(x)|\theta) = \text{MLP}_\theta \text{ where } \text{MLP}_\theta \text{ is as defined above on section "Models".}$$

Experimental setup description

We performed 3 independent experiments with the same dataset:

- **Baseline simulation:** This experiment has no monotonicity axioms except for the regression one (4.13) and establishes the base performance.
- **Locally increasing axiom:** This modifies the baseline model by adding a second axiom (4.14) which encourages the model to make a monotonically increasing prediction. The idea behind this axiom is that, despite not being absolutely true, it is quite accurate looking at the data and according to a physiological previous knowledge we have; therefore, it is not clear beforehand if it will be a misleading hypothesis or if it will actually improve the outcome.

- **Locally decreasing axiom:** Finally, our third experiment consisted on modifying the monotonicity axiom such that it encourages the outcome to be locally monotonically decreasing. We substituted the axiom (4.14) for the axiom (4.15). This new axiom is no longer an approximately accurate representation of a previous knowledge, but a completely misleading one; therefore, by keeping every other aspect of the simulation untouched, we can measure and observe how much a wrong axiom can impact the model outcome.

Results

After performing the above described experiments, we reached the following results regarding satisfaction values for both regression and monotonicity axioms as well as RMSE for both train, test and OOD data on each of the 3 experimental settings.

-	Experiment 1	Experiment 2	Experiment 3
Train Reg Sat	0.8849	0.9123	0.8848
Test Reg Sat	0.8873	0.9017	0.8790
OOD Regression Sat	0.7466	0.8019	0.7391
Train Mon Sat	-	0.9458	0.5545
Test Mon Sat	-	0.9448	0.5535
OOD Mon Sat	-	0.8632	0.6368
Train RMSE	10.6084	7.8394	10.8009
Test RMSE	10.5173	9.0493	11.0736
OOD RMSE	27.0991	20.0528	28.0119

TABLE 4.1: Results obtained after performing experiments 1 (Baseline), 2 (Favorable hypothesis) and 3 (Misleading hypothesis).

As we can see, introducing a favorable hypothesis upgraded the obtained results in comparison to the baseline. On the other hand, when instead introducing a misleading hypothesis, not only we obtained a worse result than with a favorable one (which is interesting itself since it proves that the quality of the axioms determines the quality of the outcome), but also w.r.t. the baseline. This last fact is specially interesting since we have observed that introducing a wrong axiom to the LTN not only won't upgrade its performance, but also worsen it. This is interesting since not only highlight that we should be careful when introducing axioms into a LTN, but also because this means that we can implement LTN as a mechanism for validating or refuting hypothesis by looking at how they impact the model performance.

Additionally, the degree of truth obtained on both queries (4.9) and (4.10) are:

- $\text{Sat}(\forall x, x < 1.75 \Rightarrow f(x) < 70) = 0.3725$
- $\text{Sat}(\forall x, x > 1.65 \Rightarrow f(x) > 75) = 0.9999$

As expected, the statement of every person measuring below 1.75 meters weighting below 70 lbs (32 kg) has a low truth value, since it would be true in few scenarios; whereas the statement of every person measuring more than 1.65 m weighting above 75 lbs (34 kg) has an almost 1 truth value, since it is true always or almost always.

Chapter 5

Real Life Application: Recommender System

In this section we will explore a real life application of LTN. The content of this section has been extracted from the paper Carraro et al., 2024.

On this paper, the authors aim to improve the performance of a recommender system using a LTN to reduce the data sparsity of the user-item matrix obtained when performing a vanilla approach of matrix factorization. In order to do so, the authors use knowledge on the gender preferences of each user obtained from a pre-trained model on the same data set implemented as previous knowledge. Afterwards, they implement LTN to fill the user-item matrix $\mathbf{R}$ with predicted values (whether a user likes or dislikes unseen movies) according to their genre preferences following some established logical axioms. The GitHub repository containing all the content of this paper experimental setting and code can be found on the GitHub repository of the following direction: [Recommender GitHub Repository](https://github.com/tommasocarraro/NESYKnowledgeTransfer) . URL to copy paste on your browser:

<https://github.com/tommasocarraro/NESYKnowledgeTransfer> .

Notation

Let's introduce the notation for this section, which will be as close as possible to both the previous notation and the one from the original framework. Let $\mathcal{D}$ denote the dataset, which is composed by trios of an user u , an item i and a rating r . Therefore $\mathbf{D}(u) = \text{users}$, $\mathbf{D}(i) = \text{items}$ and $\mathbf{D}(r) = \text{ratings}$; being U, I the set of users and items and $|U| = n, |I| = m$. Precisely, $\mathcal{G}(\text{ratings}) = \{0, 1\}$ since the rating system is binary, denoting 1 a liked movie and 0 a disliked one. Therefore, $\mathcal{D} \subset U \times I \times \{0, 1\}$ and we can separate it into 2 subsets $\mathcal{D} = \mathcal{D}_+ \cup \mathcal{D}_-$ where $\mathcal{D}_+$ denotes the user-item pairs with positive rate $r = 1$ whereas $\mathcal{D}_-$ is analogous with $r = 0$. Additionally we consider the set $\mathcal{D}_?$ of user-item pairs with no ratings. On the other hand, regarding the matrix factorization aspect, $\mathbf{R} \in \mathbb{N}^{n \times m}$ denotes the user-item matrix, where $\mathbf{R}_{u,i} = r$ if $(u, i, r) \in \mathcal{D}$ and 0 otherwise. Moreover, once factorized in the user-item matrix $\mathbf{R}$, the notation $\mathbf{U} \in \mathbb{R}^{n \times k}$ and $\mathbf{I} \in \mathbb{R}^{m \times k}$ denote the latent factors matrices of the users and the items, whereas $\mathbf{u}$ and $\mathbf{i}$ denote the bias vectors for users and items respectively.

Axioms, predicates and knowledge base

First of all, regarding logical operators, this paper uses the real logic product configuration.

Secondly, the axioms that constitute the Real Logic knowledge base $\mathcal{K} = \{\phi_1, \phi_2, \phi_3\}$ that the model intends to satisfy are the following:

The first axiom trivially forces the predicate Likes to be true for each positive user-item pair. This means, that any user that has ranked positively a movie, then it likes that movie.

$$\phi_1 = \forall \text{Diag}(user_+, movie_+) \text{Likes}(user_+, movie_+) \quad (5.1)$$

The second axiom is analog to the first one, but for the user who didn't like a movie.

$$\phi_2 = \forall \text{Diag}(user_-, movie_-) \neg \text{Likes}(user_-, movie_-) \quad (5.2)$$

Finally, the third axiom states that for user-items pairs in which the user hasn't rated the movie, if the movie has any genre that the user doesn't like, then the user will not like the movie.

$$\begin{aligned} \phi_3 = \forall \text{Diag}(user_?, movie_?) (\exists \text{genre} \neg \text{LikesGenre}(user_?, \text{genre}) \\ \wedge \text{HasGenre}(movie_?, \text{genre})) \Rightarrow \neg \text{Likes}(user_?, movie_?) \end{aligned} \quad (5.3)$$

Where $user_+$ and $movie_+$ are variable symbols denoting positive user-item pairs, $user_-$ and $movie_-$ negative pairs, and $user_?$ and $movie_?$ unknown rating pairs; whereas $genre$ is a variable symbol denoting the genres of the movies.

On the other, $\text{Diag}(u, i)$, $\text{Likes}(u, i)$, $\text{LikesGenre}(u, g)$ and $\text{HasGenre}(i, g)$ are the implemented predicates. $\text{Diag}(u, i)$ is the classical Diagonal predicate; $\text{Likes}(u, i)$ denotes whether a user u likes a movie i ; $\text{LikesGenre}(u, g)$ whether a user u likes a movie genre g ; and $\text{HasGenre}(i, g)$ whether a movie i belongs to genre g .

Metrics

The model maximizes the satisfaction of the knowledge base $\mathcal{K}$ of the LTN approach by minimizing the loss function:

$$L(\theta) = (1 - \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{(user_*, movie_*) \leftarrow \mathcal{B}_*}(\phi|\theta)) + \lambda \|\theta\|^2 \quad (5.4)$$

Where $* \in \{+, -, ?\}$, $\mathcal{B}_*$ denotes a batch of training examples randomly sampled from $\mathcal{D}_*$, and the notation $(user_*, movie_*) \leftarrow \mathcal{B}_*$ denotes that variables $user_*$ and $movie_*$ are grounded with actual user-movie pairs coming from the corresponding batch $\mathcal{B}_*$.

On the other hand, when performing the training of the MF baseline, the objective function to minimize is the following:

$$\frac{1}{N} \sum_{(u,i,r) \in \mathcal{D}} -\alpha_t (1 - p_t)^\gamma \log(p_t) + \lambda \|\theta\|^2, \quad (5.5)$$

$$\text{where } \alpha_t = \begin{cases} \alpha & \text{if } r = 1, \\ 1 - \alpha & \text{if } r = 0 \end{cases} \text{ and } p_t = \begin{cases} p & \text{if } r = 1, \\ 1 - p & \text{if } r = 0 \end{cases}$$

And $p = \sigma(\mathbf{U}_u \cdot \mathbf{I}_i^\top + \mathbf{u}_u \cdot \mathbf{i}_i)$, where σ is the logistic function.

When evaluating the performance of the model, the authors measured precision, recall, and F1-measure.

Experiments

Datasets

The selected dataset is MindReader (MR) Brams et al., 2020 <https://mindreader.tech/dataset/>, a novel dataset containing ratings from real users for movies and non-recommendable entities, such as movie genres. Precisely, there are two versions of this dataset (MR-100k and MR-200k), and all experiments have been performed on both versions.

Pre-processing has been applied to both datasets, but since discussing it is out of the scope of this chapter, we encourage anyone interested to look up the original document Carraro et al., 2024.

Experimental Setup description

The studied paper compares this LTN Neuro-symbolic approach (denoted as NESY_{MF}) with a baseline MF model. This two models are both trained on both the full data set and on additional training sets generated from randomly sampling the 50%, 20%, 10%, and 5% of the movie rating from the entire training set to compare the obtained results on all these scenarios between them. The idea behind this approach is measuring how much of an improvement means implementing this knowledge transfer technique depending on the sparsity of the data. The entire procedure is repeated 30 times using seeds from 0 to 29 and the results have been averaged and are shown on table (5.1).

All models were trained on 500 epochs using the Adam optimizer and early stopping if no improvements were found on the validation set for 10 epochs. $\mathbf{U}$, $\mathbf{I}$, $\mathbf{u}$ and $\mathbf{i}$ were randomly initialized using the *Glorot* initialization.

The original paper also searched for the best values for the hyper-parameters k (number of latent factors), λ (regularization coefficient), η (learning rate), β (training batch size), α (focal loss hyper-parameter), t (decision boundary threshold for the MF model) and p (A_{pM} and A_{pME} p hyperparameter for the LTN model) through Bayesian optimization. However, since we are not interested on this aspect of the work and results are only presented for the best found values, we encourage any interested reader to consult the original framework Carraro et al., 2024.

Results

The results obtained by the authors on the original paper are presented on Table (5.1).

As it can be seen, despite worsening precision on an average value around 0.01, implementing the LTN knowledge transfer technique significantly improves F1-measure and, specially, recall. Moreover, we can observe that, the less data there is available (the more the dataset is sparse), the bigger improvement it achieves on recall.

In particular, this recall increase completely makes sense looking at the knowledge base that the model has learned. If we look closely at the axiom (5.3), it aims to discard as many possible "bad" recommendations (understanding bad recommendations as recommended items that the user ended up disliking) since it doesn't consider any movie that is suspicious of not being liked by the user (since having any genre that he/she dislikes is reason enough for avoiding it). Therefore, the model ends up focusing on safer and more confident recommendations out of the immense pool of recommendable items, achieving a higher True Positive Rate (TPR) i.e. recall.

(a) MR-100k				(b) MR-200k			
Fold	Metric	MF	NESY _{MF}	Fold	Metric	MF	NESY _{MF}
100%	Precision	0.5595 _(0.0592)	0.5432 _(0.0559)	100%	Precision	0.5640 _(0.0560)	0.5531 _(0.0503)
	Recall	0.7377 _(0.0452)	0.7848 _(0.0395)		Recall	0.7555 _(0.0333)	0.7931 _(0.0207)
	F1-measure	0.6328 _(0.0322)	0.6392 _(0.0349)		F1-measure	0.6431 _(0.0341)	0.6498 _(0.0334)
50%	Precision	0.5410 _(0.0580)	0.5297 _(0.0512)	50%	Precision	0.5510 _(0.0554)	0.5415 _(0.0491)
	Recall	0.6723 _(0.0459)	0.7054 _(0.0260)		Recall	0.6893 _(0.0332)	0.7240 _(0.0195)
	F1-measure	0.5956 _(0.0276)	0.6030 _(0.0307)		F1-measure	0.6095 _(0.0300)	0.6178 _(0.0306)
20%	Precision	0.5081 _(0.0558)	0.4946 _(0.0532)	20%	Precision	0.5093 _(0.0499)	0.5050 _(0.0474)
	Recall	0.5948 _(0.0411)	0.6529 _(0.0249)		Recall	0.5838 _(0.0302)	0.6459 _(0.0171)
	F1-measure	0.5444 _(0.0261)	0.5605 _(0.0311)		F1-measure	0.5413 _(0.0231)	0.5650 _(0.0286)
10%	Precision	0.4856 _(0.0545)	0.4689 _(0.0545)	10%	Precision	0.4806 _(0.0469)	0.4725 _(0.0441)
	Recall	0.5468 _(0.0357)	0.6456 _(0.0322)		Recall	0.5359 _(0.0283)	0.6067 _(0.0193)
	F1-measure	0.5112 _(0.0261)	0.5407 _(0.0349)		F1-measure	0.5041 _(0.0202)	0.5295 _(0.0258)
5%	Precision	0.4606 _(0.0476)	0.4474 _(0.0522)	5%	Precision	0.4508 _(0.0438)	0.4422 _(0.0428)
	Recall	0.5284 _(0.0250)	0.6584 _(0.0286)		Recall	0.5049 _(0.0257)	0.5953 _(0.0203)
	F1-measure	0.4897 _(0.0213)	0.5304 _(0.0358)		F1-measure	0.4740 _(0.0193)	0.5057 _(0.0273)

FIGURE 5.1: Comparison of averaged results of MF and NESY_{MF} on the selected datasets.

We could start a debate on whether this is or this isn't a desirable effect to be achieved, since maybe we want our model to perform not only safe recommendations; however, this demonstrates that LTN have a huge potential as a knowledge transfer technique, since the model has shown to achieve the goal for which it was designed and, specifically, it has performed specially well on the scenarios in which it was required the most (less data, bigger need of knowledge transfer).

Chapter 6

Conclusions

On this thesis we have presented a theoretical and practical treatment of Logic Tensor Networks. Firstly, on chapter 3 we provided a description of LTN, focusing on the Real Logic language they use, the grounding mechanism of their elements, the logical operators and their learning mechanism and architecture; whereas on chapters 4 and 6 we presented practical applications of them. First, we developed 3 simple experiments on a controlled environment to understand the shape that has each component of the LTN and how they work. On the other hand, we presented a real-life application of a LTN to show their potential and application on a less simple scenario. On these two chapters we demonstrated the utility and potential of LTN, complementing the theoretical explanation provided on the previous chapter.

Out of this thesis and observed results, we can extract several conclusions. First of all, LTN arise as a successful knowledge transferring mechanism that allows to translate logical statements into actual differentiable, and therefore optimizable, functions; enabling us as humans to define constraints on our human logical language. Secondly, we have verified that accurate hypothesis improve model's performance, whereas misleading ones can decrease it. Thirdly, we observed that LTN's provide us a new explainability level on the output predictions; not only measuring how well (satisfaction degree) a prediction each axiom, but also allowing us to measure the degree of truth of some logical statement through querying. To sum up, LTN have proved their potential and usefulness on multiple tasks regarding knowledge transfer.

Finally, we would like to point out some future work lines that arise from this thesis. First of all, we could explore new and more ambitious applications of LTN, not restricting ourselves just to some simple regressor; such as the recommender use case. Secondly, taking this particular case, we could experiment with it modifying the knowledge base (i.e. modifying the third axiom) to observe how performance and results vary depending on the design intention behind the new axioms. Moreover, regarding theoretical aspects, we could attach reasoning through LTN, as well as discussing different ways to represent knowledge.

If any lector is interested on checking the code of the performed experiments, you can enter the following GitHub repository: [My GitHub Repository](https://github.com/srguiot14/Logic-Tensor-Networks-for-Knowledge-Infused-Learning-Foundations-Experiments-and-Applications). URL to copy paste on your browser: <https://github.com/srguiot14/Logic-Tensor-Networks-for-Knowledge-Infused-Learning-Foundations-Experiments-and-Applications>

Bibliography

- Badreddine, Samy et al. (2020). “Logic Tensor Networks”. In: *CoRR* abs/2012.13635. arXiv: 2012.13635. URL: <https://arxiv.org/abs/2012.13635>.
- Brams, A. H. et al. (2020). “Mindreader: Recommendation over knowledge graph entities with explicit user ratings”. In: *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. DOI: 10.1145/3340531.3412759.
- Carraro, Tommaso et al. (2024). “Mitigating Data Sparsity via Neuro-Symbolic Knowledge Transfer”. In: *Advances in Information Retrieval*. Ed. by Nazli Goharian et al. Cham: Springer Nature Switzerland, pp. 226–242. ISBN: 978-3-031-56063-7.
- Serafini, Luciano, Ivan Donadello, and Artur d’Avila Garcez (2017). “Learning and reasoning in logic tensor networks: theory and application to semantic image interpretation”. In: *Proceedings of the Symposium on Applied Computing*. SAC ’17. Marrakech, Morocco: Association for Computing Machinery, 125–130. ISBN: 9781450344869. DOI: 10.1145/3019612.3019642. URL: <https://doi.org/10.1145/3019612.3019642>.
- Valiant, Leslie (Mar. 2000a). “Robust logics”. In: *Artificial Intelligence* 117, pp. 231–253. DOI: 10.1016/S0004-3702(00)00002-3.
- (Jan. 2006). “Knowledge Infusion.” In: vol. 2.
- Valiant, Leslie G. (Sept. 2000b). “A neuroidal architecture for cognitive computation”. In: *J. ACM* 47.5, 854–882. ISSN: 0004-5411. DOI: 10.1145/355483.355486. URL: <https://doi.org/10.1145/355483.355486>.